Frontend : React + Tailwind

Backend : Spring Boot

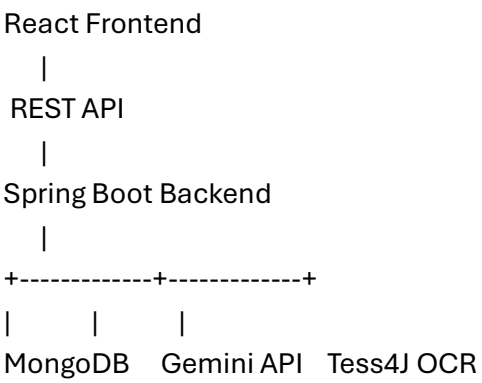
Database : MongoDB

AI : Gemini API

OCR : Tess4J

then here is a complete project structure.

Overall Architecture



Project Modules

1. Authentication
2. Seller Management
3. Document Upload
4. OCR Extraction
5. Product Catalog Generation
6. Compliance Checker
7. Dashboard
8. ONDC Export Module

Frontend Structure (React)

frontend/

src/

```
├── assets
│
├── components
│   ├── Navbar.jsx
│   ├── Sidebar.jsx
│   ├── UploadCard.jsx
│   ├── ProductCard.jsx
│   ├── ComplianceCard.jsx
│
├── pages
│   ├── Login.jsx
│   ├── Register.jsx
│   ├── Dashboard.jsx
│   ├── SellerProfile.jsx
│   ├── UploadDocuments.jsx
│   ├── ProductUpload.jsx
│   ├── Catalog.jsx
│   ├── Compliance.jsx
│
├── services
│   ├── authService.js
│   ├── sellerService.js
│   ├── productService.js
│   ├── documentService.js
│
├── routes
│   └── AppRoutes.jsx
│
├── context
│   └── AuthContext.jsx
│
├── App.jsx
│
└── main.jsx
```

Backend Structure (Spring Boot)

backend/

src/main/java/com/ondc/

```
├── controller
│
├── service
│
├── repository
│
├── entity
│
├── dto
│
├── config
│
├── exception
│
├── util
│
├── ai
│
└── ONDCAutoOnboarderApplication.java
```

Controllers

controller/

AuthController.java

SellerController.java

DocumentController.java

OCRController.java

ProductController.java

ComplianceController.java

DashboardController.java

Services

service/

AuthService.java

SellerService.java

DocumentService.java

OCRService.java

GeminiService.java

ProductService.java

ComplianceService.java

DashboardService.java

MongoDB Collections

Sellers


```
{
  "_id":"1",
  "businessName":"ABC Traders",
  "ownerName":"Ramesh",
  "email":"abc@gmail.com",
  "phone":"9876543210",
  "gstNumber":"29ABCDE1234A1Z5",
  "address":"Hyderabad"
}
```

Documents


```
{
  "_id":"2",

  "sellerId":"1",

  "documentType":"GST",

  "filePath":"uploads/gst.pdf",

  "ocrText":"GST Number: 29ABCDE1234A1Z5"
}
```

Products


```
{
  "_id":"3",

  "sellerId":"1",

  "productName":"Sports Shoe",

  "description":"Lightweight running shoe",

  "category":"Footwear",

  "imageUrl":"uploads/shoe.jpg"
}
```

Compliance


```
{
  "_id":"4",

  "sellerId":"1",

  "score":95,
```

```
 "status":"ONDC Ready",

 "remarks":[
    "Description valid",
    "Category valid"
 ]
}
```

Entity Classes

Seller

```
@Document(collection="sellers")
@Data
public class Seller {

   @Id
   private String id;

   private String businessName;
   private String ownerName;
   private String email;
   private String phone;
   private String gstNumber;
   private String address;
}
```

Product

```
@Document(collection="products")
@Data
public class Product {

   @Id
   private String id;

   private String sellerId;

   private String productName;
```

```
    private String description;

    private String category;

    private String imageUrl;
}
```

APIs

Authentication

POST /api/auth/register

POST /api/auth/login

Seller

POST /api/seller

GET /api/seller/{id}

PUT /api/seller/{id}

Documents

POST /api/document/upload

GET /api/document/{sellerId}

OCR

POST /api/ocr/extract

Response

```
{
  "businessName":"ABC Traders",
  "gstNumber":"29ABCDE1234A1Z5"
}
```

Product Catalog

POST /api/product/generate

Input

```
{
  "imageUrl":"shoe.jpg"
}
```

Gemini Output

```
{
 "title":"Sports Shoe",
 "description":"Comfortable running shoe",
 "category":"Footwear"
}
```

Compliance

POST /api/compliance/check

Output

```
{
  "score":95,
  "status":"ONDC Ready"
}
```

AI Folder

ai/

GeminiService.java

PromptBuilder.java

CatalogGenerator.java

ComplianceAI.java

OCR Folder

util/

OCRUtil.java

FileUploadUtil.java

OCR Flow:

GST Certificate
↓
Upload
↓
Tess4J OCR
↓
Extract Text
↓
Regex Extraction
↓
Store in MongoDB

Dashboard Flow

Seller Registered
↓

Documents Uploaded
↓

OCR Extraction Complete
↓

Product Images Uploaded
↓

Gemini Generates Catalog
↓

Compliance Check
↓

ONDC Ready Status

Database Relationships

Seller
|
+------ Documents
|
+------ Products
|
+------ Compliance

Store only the sellerId in MongoDB documents.

Development Roadmap

Week 1

React Setup
Spring Boot Setup
MongoDB Connection
Authentication
Seller Module

Week 2

File Upload
OCR Integration
Document Management

Week 3

Gemini Integration
Catalog Generator
Compliance Checker

Week 4

Dashboard
Testing
PPT
Demo Preparation

This structure is large enough to look like an industry-level project while still being achievable by a student team using React + Spring Boot + MongoDB.

i am confused about teh ai and ocr ,how to do them ,

Don't worry about the AI and OCR part. Most student teams get stuck here because they think they need to build their own AI model.

You do NOT need to train any AI model.

For your project:

- OCR = Use an existing library (**Tess4J/Tesseract**)
- AI = Call an existing model (**Gemini API**)

Think of them as external services.

Part 1: OCR (Easy)

Goal

User uploads:

GST Certificate.pdf

Your backend extracts:

Business Name : ABC Traders
GST Number : 29ABCDE1234A1Z5
Address : Hyderabad

How OCR Works

PDF/Image
↓
Tess4J
↓
Extracted Text
↓
Regex
↓
GST Number Found

Example GST certificate:

ABC Traders Pvt Ltd

GSTIN : 29ABCDE1234A1Z5

Address:
Hyderabad

Tess4J converts image → text.

Spring Boot OCR Example

Dependency:

```
<dependency>
  <groupId>net.sourceforge.tess4j</groupId>
  <artifactId>tess4j</artifactId>
  <version>5.8.0</version>
</dependency>
```

Service:

```
@Service
public class OCRService {

  public String extractText(File file) throws Exception {

    Tesseract tesseract = new Tesseract();

    tesseract.setDatapath("tessdata");

    return tesseract.doOCR(file);
  }
}
```

Output:

```
ABC Traders Pvt Ltd

GSTIN : 29ABCDE1234A1Z5

Hyderabad
```

Extract GST Number

Use Regex:

```
Pattern pattern =
Pattern.compile("\\d{2}[A-Z]{5}\\d{4}[A-Z]{1}[A-Z\\d]{1}[Z]{1}[A-Z\\d]{1}");
```

```
Matcher matcher = pattern.matcher(text);

if(matcher.find()){
   gstNumber = matcher.group();
}
```

Result:

29ABCDE1234A1Z5

Part 2: AI (Much Easier Than OCR)

You don't build AI.

You send a prompt to Gemini.

Gemini sends back the answer.

Example

User uploads:

shoe.jpg

You ask Gemini:

Generate:

1. Product Title
2. Product Description
3. Category

for a sports shoe.

Gemini returns:

```
{
  "title":"Sports Shoe",

  "description":"Lightweight running shoe with breathable mesh.",

  "category":"Footwear"
}
```

Save it to MongoDB.

Done.

Gemini Integration

Dependency:

```
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
</dependency>
```

Use Spring's RestTemplate or WebClient.

Simple Flow

Product Image Upload
↓
Backend receives image
↓
Send prompt to Gemini
↓
Gemini generates catalog
↓
Store in MongoDB

What AI Should Actually Do?

You only need 3 AI features.

Feature 1

Catalog Generation

Input:

Sports Shoe

Output:

Title
Description
Category

Feature 2

Compliance Suggestions

Input:

Title: Shoe
Description: Nice shoe

AI says:

Title too short.
Add material details.

Feature 3

Business Summary

Input:

Business Name
GST Details
Products

Output:

Seller profile generated.

The Simplest Approach

Instead of image understanding at first, do this:

User enters:

Product Name:
Sports Shoe

Then Gemini generates:

Description
Category
Features

This is much easier.

Later, if time permits, add image-based generation.

What You Should Build First

Phase 1

React
Spring Boot
MongoDB

Make CRUD APIs.

Phase 2

Document Upload
OCR Extraction

Use Tess4J.

Phase 3

Gemini API
Catalog Generator

Send prompts.

Phase 4

Compliance Checker
Dashboard

My suggestion

For the project demo, don't start with AI image recognition.

Start with:

Upload GST PDF
↓
OCR extracts GST number
↓
Enter Product Name
↓

Gemini generates Description

↓

Compliance Score

↓

Dashboard

This is much easier to complete and still fully demonstrates **AI-powered onboarding automation**.